Problem Set 1

Problem 4

Find the complexity of the below functions

```
int function0(int n) {
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    return sum;
}
void function1(int n)
{
      for (int i=1; i<n; i=i*2)

      printf("Marhaba");
}
int function2(int n)
{
      int c = 0;
      for (int i=n/2; i<=n; i++)
            for (int j=1; j<=n; j++)
                  c+=i+j;

return c ;
}
void function3(int n)
{
      int count = 0;
      for (int i=1; i<=n/3; i++)
            for (int j=n/2; j<=n; j++)
                  for (int k=1; k<=n; k
= k * 2)
                        count++;

}
void function4(int n)
{
      int count = 0;
      for (int i=n/2; i<=n; i++)
            for (int j=1; j+n/2<=n; j =
j++)
                  for (int k=1; k<=n; k
= k * 2)
                        count++;

}
void function5(int n)
{
      int i = 1, s =1;
      while (s <= n)
      {
            i++;
            s += i;
            printf("*");
      }
}
```

```
int function6 (int n) {
    int r, d;
    d = 1;
    r = 0;

    while (d * d <= n) {
        d = d * 2;
    }

    while (d > 1) {
        d = d / 2;
        if ((r + d) * (r + d) <= n) {
            r = r + d;
        }
    }

    return r;
}

void Function7(int n, int m, int *q, int
*r) {
    *q = 0;
    *r = n;
    while (m <= *r) {
        (*q)++;
        *r -= m;    }

}
```

Problem 5

Give two functions with different complexities to check if an integer n is prime number

Problem 6

Give a function to count the number of digits in a positive integer, then estimate its complexity

Problem 7

Propose programs for the following problems, and then calculate their complexities:

- Rotate an n x n matrix in the clockwise
- Transpose an n x m matrix A into an m x n matrix B.
- Multiply two matrices A (n x m) and B (m x p) to get C (n x p).

University of Mascara
Department of Computer sciences
Second Year Engineer

Algorithmic and Complexity

Problem Set 1

Problem 1

For each function f(n) and time t in the following table, determine the largest size n of a problem that can be solved in time t, assuming that the algorithm to solve the problem takes f(n) *microseconds*.

f(n) \ t	1 second	1 minute	1 hour	1 day	1 month	1 year
log(n)						
$\sqrt{n}$						
n						
n log(n)						
n^2						
2^n						
n!						

Problem 2 Arrange the following functions in ascending order of complexity

n, 2000, $\sqrt{n}$, $\log^2$ (n), n$\sqrt{n}$, log (log(n)) , $\sqrt{n}$log(n), n/log(n)

Problem 3 Express each function in terms of big O notation and determine its complexity class (the symbol ^ means power)

5n + 3, 3n^2 + 2n – 1, 2n^3 + 4n^2 - 7n, 10n^2 + 8n + 12, 2^n + 3n, n!, n^4 + 2n^3 + n^2 + n + 1, log(2n+1), √5n+1, (5n+1) log(n/2), 4n^5 - 3n^3+7n, 2^n + n^4, n^(log(3)), 2n^2 + 3n log2(n), n^(n+1)